rOpenSci Localization and Translation Guidelines

Yanina Bellini Saibene, Paola Corrales, Elio Campiteli, Maëlle Salmon

Table of contents

Preface **4**
Welcome! . . . 4
Intended Audience . . . 4
What You Will Learn . . . 5
Contributing and Re-Use . . . 5
Attributions . . . 6

1 Motivation **7**
1.1 Language barrier . . . 7
1.2 Community driven localizations and translations . . . 8
1.3 Building community . . . 9
1.4 References . . . 9

2 General Guidelines **10**
2.1 Introduction . . . 10
2.2 Technical infrastructure and organization of the work . . . 11
2.3 General aspects of the translation process . . . 12
2.4 Using the localizations materials . . . 13
2.4.1 Citing a translation . . . 13
2.4.2 Mention your contribution to a localization project in your CV . . . 14
2.5 References. . . . 14

3 Translation and Review Guidelines **15**
3.1 How to contribute . . . 15
3.1.1 With an active language . . . 15
3.1.2 With a new language . . . 15
3.2 Technical aspects of the review . . . 16
3.2.1 Review 1 . . . 16
3.2.2 Review 2 . . . 18
3.2.3 Pull Request and editing of a chapter . . . 20
3.3 General review . . . 23

4 Specific Guidelines **24**
4.1 Technical terms and citations . . . 24
4.2 Code . . . 24
4.3 Diagrams . . . 25

5 NEWS **26**

Preface

Welcome!

This book explains all the steps and tools involved in the localization of our materials, including translations. We design, build, and test a process for maintain the different languages version of our materials that take ideas from our software peer review system and the maintenance of open-source projects.

We know that code peer review ensure the quality of our software; with the same goal of ensuring the quality of our content in different languages, we implement a system to review and maintain the localization of our guides, books, blog post, and other content.

We also know that by using roles (such as reviewers and maintainers) and tools (such as GitHub and Pull Request) known to our community, we make this process easier to undestand, apply and, therefore, easier to contribute to.

Intended Audience

This book is a guide for people who would like to *localize rOpenSci materials* or *contribute to maintaining localized material*. It is also intended for people that have to localize material on other community or group. You don't need to be a proffesional translator or developer to contrinute to this effort. The following personas are examples of the types of people that are our target audience.

Ana is a Colombian student doing her Ph.D. at a university in Argentina. She works with geospatial data. She is not an expert in programming, but she took one of The Carpentries workshops for scientists and used some of the rOpenSci suite packages for handling and accessing geospatial data for her thesis. Language was one of the barriers she faced when learning to code. Ana wants to contribute by translating material into her native language so others can access this knowledge with less effort. This book will show her how to do a new translation and how to review an existing one.

Francesco will send their packages to the rOpenSci review process in Spanish. They know that this will ensure the quality of their software and will also facilitate its publication in JOSS, which will give them academic credit for their job at their university. They read the Spanish version of the book "rOpenSci Packages: Development, Maintenance,

and Peer Review" to prepare their submission. If their review experience is good, they will volunteer to be a reviewer in their native language and contribute to the book. This guide will explain how to become a contributor and maintainer of translated material and the infrastructure that allows translations.

Bauti is the co-founder of a user group in his city in Brazil. With a couple of university classmates, he teaches and learns together how to code and do reproducible analysis using different languages. He uses rOpenSci Spanish material during his workshops and would like to have the material from other programming languages in Spanish. He wonders how rOpenSci manages translation. This book will show him how we organize translation and what tools we use so that he can apply in his community.

What You Will Learn

The guide contains:

- Why we localize, translate and work to do multilingual publishing.
- General guide with the infrastructure and roles, step-by-step technical process, and localization considerations, including the technical terms to translate and the technical terms we will not translate.
- Language-specific guides with localization considerations.
- Language-specific glossary.

At rOpenSci, we use R as our main programing language and GitHub to host our code, including our packages, books, webpage and review process. The step-by-step technical process refers to working with GitHub, like making pull requests, reviewing and commenting on pull requests, and creating issues. It also explains how to use the {babeldown} R package that facilitates working with translations.

The localization consideration and the glossary are language-specific and detail the agreements the community reach about how to localize and translate the material. This chapter is different for each language present in this guide.

We hope that you'll find the guide useful and clear, and welcome your suggestions in the issue tracker of the book. Please check Contributing and Re-Use section to learn more about hot to contribute.

Happy localization!

Contributing and Re-Use

This book is a living document. If you want to contribute to this book, corrections, additions, and suggestions are very welcome. Everyone whose work is included will be credited

in the acknowledgments. Please refer to the GitHub repository, particularly the contributing guidelines and our Code of Conduct, for more information on how to contribute.

You can cite this book:

rOpenSci Localization and Translation Guidelines v1.0. Yanina Bellini Saibene, Paola Corrales, Elio Campiteli, Maëlle Salmon (November, 2023)

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 License.

Thanks!

You can also read the PDF version of this book.

Attributions

These translation guidelines are based on the experience and documents generated during the translation of Teaching Tech Together.

1 Motivation

At rOpenSci we work to transform science through open data, software and reproducibility. We provides community support, standards, and infrastructure for scientists and research software engineers working in R to develop, maintain, and publish high-quality open-source scientific software. In addition, we develop and maintain high-quality documentation and resources to support these activities.

We also believe that science and the research software we create to support it, should serve everyone in our communities, which means it needs to be sustainable, open, and built by and for all groups Currently, however, there is a dismaying lack of diversity in scientific and open source communities in general. That disappointing lack of diversity is potentially detrimental to the sustainability, utility and productivity of projects.

rOpenSci is carrying out a series of activities and projects to transform science to ensure that serves everyone in our communities which include our multilingual publishing project.

1.1 Language barrier

English is the *lingua franca* for science creating a significant barrier for non-English speaker waiting to join the field. It is also the predominant language of open source in code, content, and community interactions making *language access* one of the *environmental barriers to equity in open source.*[1] .

The UNESCO's recommendation on open science highlights the need to overcome language barriers in order to achieve the open science core values and guiding principles of Equity and fairness, Diversity and inclusiveness, Equality of opportunities, and Collaboration, participation and inclusion[2].

A recent study[3] quantified the consequences of language barriers on the career development of researchers. The authors found that Non-native English researchers need:

[1] Hilary Carter and Jessica Groopman, "Diversity, Equity, and Inclusion in Open Source: Exploring the Challenges and Opportunities to Create Equity and Agency Across Open Source Ecosystems", foreword by Jim Zemlin, The Linux Foundation, December, 2021, https://www.linuxfoundation.org/research/the-2021-linux-foundation-report-on-diversity-equity-and-inclusion-in-open-source

[2] UNESCO. UNESCO recommendation on open science. Paris, France; 2021.

[3] Amano T, Ramírez-Castañeda V, Berdejo-Espinola V, Borokini I, Chowdhury S, Golivets M, et al. (2023) The manifold costs of being a non-native English speaker in science. PLoS Biol 21(7): e3002184. https://doi.org/10.1371/journal.pbio.3002184

- up to **91% more time to read** English papers,
- up to **51% more time to write** papers and their papers will be **rejected 2.6 times more often**. If accepted, they'll have to **revise it 12.5 more times**
- When they present in English need up to **94% more time** to prepare their presentations. They will **often avoid** English speaking conferences and oral presentations.

Furthermore, the Linux Foundation report states, "English proficiency is a metric by which performance and personality can be judged"[4].

These studies also higligh that "the magnitude of the disadvantage seems far beyond the level that can be overcome with individuals' efforts."[5]

By developing the technical and social infrastructure to publishing multilingual resources and publish our own resources in several languages, we can lower these barriers by *increasing access to knowledge* and *democratize access to quality resources*, thereby *increasing the potential for individuals to contribute* to software and open science projects.

1.2 Community driven localizations and translations

Community-driven translations and localizations are efforts by community members to have a resource in a language of their choice. The community organized and agreed on different aspects of the localization project[6].

The Spanish-speaking R community has been very active and growing in recent years and has undertaken various translation activities for technical materials such as books, cheat sheets, guides, and datasets.

In 2017, several Latin American R-Ladies began translating the R-Ladies' Code of Conduct and their Rules and Guidelines into Spanish.

In 2018, the R community in Latin America collectively translated the R for Data Science book into Spanish. This included the translation of all the data sets used in the book, which were compiled in the datos package, making it an excellent tool for teaching.

[4] Hilary Carter and Jessica Groopman, "Diversity, Equity, and Inclusion in Open Source: Exploring the Challenges and Opportunities to Create Equity and Agency Across Open Source Ecosystems", foreword by Jim Zemlin, The Linux Foundation, December, 2021, https://www.linuxfoundation.org/research/the-2021-linux-foundation-report-on-diversity-equity-and-inclusion-in-open-source

[5] Amano T, Ramírez-Castañeda V, Berdejo-Espinola V, Borokini I, Chowdhury S, Golivets M, et al. (2023) The manifold costs of being a non-native English speaker in science. PLoS Biol 21(7): e3002184. https://doi.org/10.1371/journal.pbio.3002184

[6] Yanina Bellini Saibene and Natalia Soledad Morandeira. Multilingual Data Science: Ten Tips to Translate Science and Tech Content. Chapter at Our Environment.A collection of work by data designers, artists, and scientists. ISBN:979-8-218-20191-3. http://datasciencebydesign.org/blog/multilingual-data-science

The community continued with the translation of Teaching Tech Together and contributed to the translations into Spanish of the Posit Cheatsheets, The Carpentries' and The Programming Historian lessons.

Driven by this active and growing Spanish-speaking community, rOpenSci successfully piloted our first Spanish-language peer review, where the submission, reviews, and editorial responses were in Spanish.

These works and previous community experiences created the conditions for us to start our multilingual publishing in Spanish. Spanish is the second most-spoken native language in the world and is one of the most geographically widespread languages, being an official language in many countries[7].

1.3 Building community

At rOpenSci, we understand review as a way of building community and have our review process in Spanish will allow us to continue building the community in Spanish-speaking regions, to increase the number of contributors, and to get feedback on how our tools and processes can be improved to better serve these community members.

In addition to use our material to learn how to be a developer, maintainer, reviewer, or editor, people can also contribute through translation. This type of contribution is often a good first step and is recognized as valuable by the community.

We also expect that the project's documentation and tooling will be useful for extending this effort to other languages and for other communities and projects with translation efforts.

1.4 References

[7] List of languages by number of native speakers. Accessed on December 1, 2022. https://en.wikipedia.org/wiki/List_of_languages_by_number_of_native_speakers

2 General Guidelines

> "Meet your community member where they are."

2.1 Introduction

rOpenSci provides community support, standards, and infrastructure for scientists and research software engineers working in R to develop, maintain, and publish high-quality open-source scientific software. In addition, we develop and maintain high-quality documentation and resources to support these activities. Many of our materials are defined as *living documents*, meaning that they are constantly being improved and updated.

Multilingual publishing of documentation and resources involves two aspects:

- The *internationalization* refers to the technological solution that allows the software to adapt to different regions without requiring engineering changes to the source code. It is what enables us to localize our content[1].
- The *localization* is the process of taking a piece of content in its original form and converting it into something more accessible and suitable for another region, country, or audience[2]. This includes language, date formats, currency, measurement units, and support for different character sets.

In addition, the localization of living documents has two well-defined stages involving different resources:

1. achieving a first version of the translated material and
2. keeping the material updated and synchronized between the supported languages[3].

Translation is typically the most time-consuming component of these efforts[4].

[1] Internationalization and localization. Access November 1, 2022. https://en.wikipedia.org/wiki/Internationalization_and_localizat
[2] Internationalization and localization. Access November 1, 2022. https://en.wikipedia.org/wiki/Internationalization_and_localizat
[3] Yanina Bellini Saibene and Natalia Soledad Morandeira. Multilingual Data Science: Ten Tips to Translate Science and Tech Content. Chapter at Our Environment.A collection of work by data designers, artists, and scientists. ISBN:979-8-218-20191-3.
[4] Internationalization and localization. Access November 1, 2022. https://en.wikipedia.org/wiki/Internationalization_and_localizat

2.2 Technical infrastructure and organization of the work

There are many solutions and tools for internationalizing and localizing content and software. For example, translation management systems (Crowdin, Transifex, Weblate), automatic translators (Google Translate, DeepL), version control systems (GitHub, GitLab), markup languages (LaTeX, Markdown), and tools for writing these languages (Overleaf, Quarto).

These technological solutions are continuously evolving. We developed our infrastructure for our localization effort using the tools that best suit our team, contributors and materials, and, in summary, our community. The choice of technology impacts the paths we create for contributions, so they should not be a barrier[5].

We also developed - and documented in these guidelines - a workflow that follows the same idea of using the tools our community already knows and uses in their activities. We use GitHub projects to track each localization project's progress and the people who have different contribution roles, like *reviewers*, *editors,* and *maintainers*.

[5] Yanina Bellini Saibene and Natalia Soledad Morandeira. Multilingual Data Science: Ten Tips to Translate Science and Tech Content. Chapter at Our Environment.A collection of work by data designers, artists, and scientists. ISBN:979-8-218-20191-3.

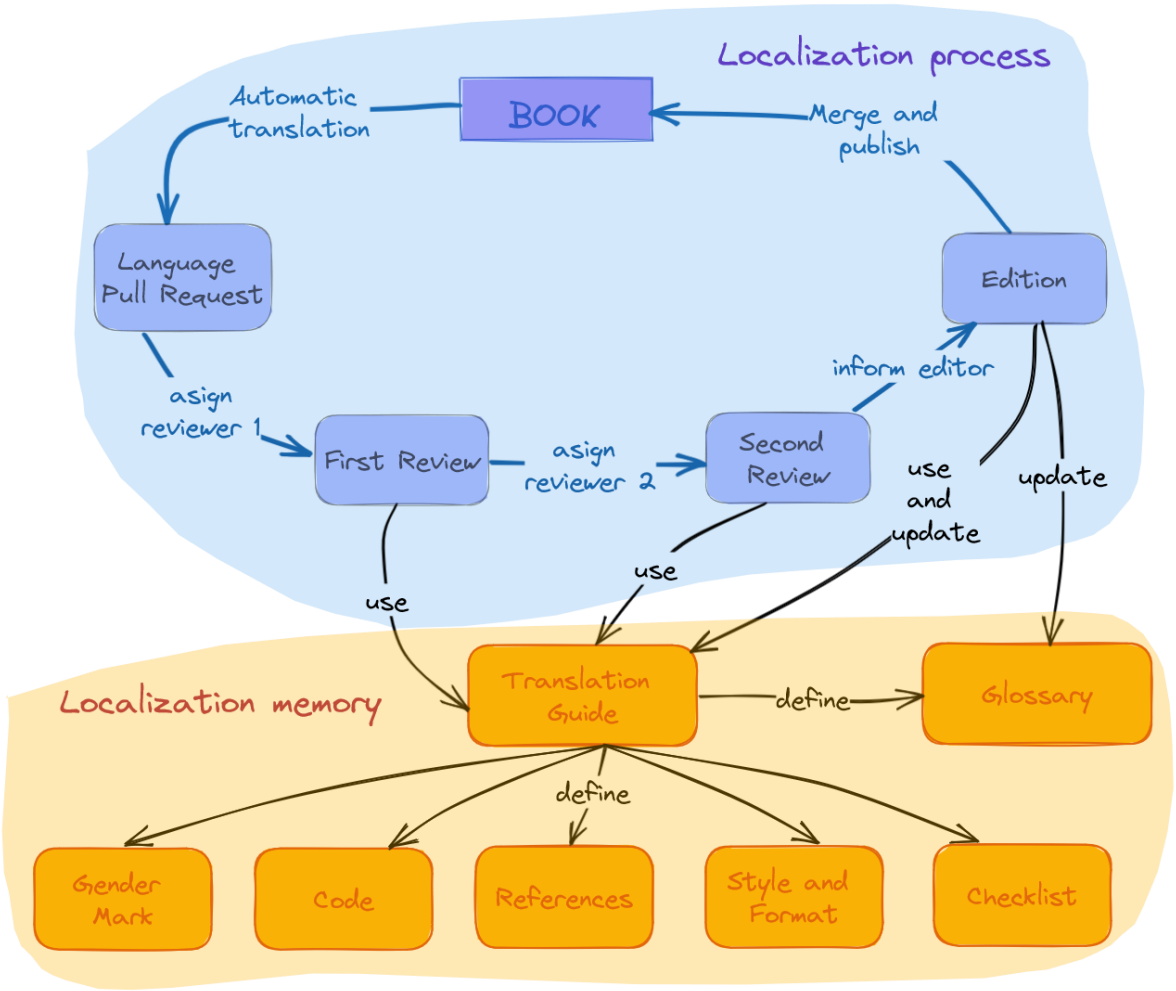


Figure 2.1: rOpenSci Localizacion Process

2.3 General aspects of the translation process

The translation process starts with a first machine translation using the babeldown package. This provides a first draft that is then reviewed by human eyes, who correct errors and incorporate the localization and language translation agreements (see Languaje specific guidelines).

To minimize errors and maintain a broad look at the translation, we at rOpenSci ask that each chapter or section goes through at least two reviews done in series (the first one reviews the automatic translation and the second reviews the first review) followed by a general review and edition of the book or document as a whole.

In each step of this process, we ask you to:

1. Use a conversational voice rather than a formal or academic voice.
2. If appropriate, specify the dialect or regional language variation used. For example, the Spanish translation uses Latin American conventions.
3. Try to be gender-neutral. If the language you are working on has a strong grammatical gender, the translation adjusts wording to avoid assigning a gender. Where a gender mark cannot be avoided, it uses feminine-masculine or masculine-feminine splits. For consistency throughout the text and to show that there is no particular hierarchy, it alternates the use of feminine or masculine between chapters, with the use being consistent throughout each chapter.
4. Try to be idiomatic. Don't be literal, and bring the message closer to your audience.[6] When translating, the text and the form that best expresses the meaning of the fragment in the target language should be prioritized, not the one that seems to be literal from the original.

The translation and review process is done on GitHub using *pull requests* (see this section for details). We chose this workflow as this is the infrastructure we use in our community. All our packages and books are hosted on GitHub and use *issues* and *pull requests* in their development. In addition, this allows the process to be open so that others can contribute and provide feedback.

Of course, as in all areas of rOpenSci, we follow our code of conduct to create a friendly and safe environment.

2.4 Using the localizations materials

All the next examples use the rOpenSci Packages: Development, Maintenance, and Peer Review localization, but you can apply this recommendations to any of the other translations.

2.4.1 Citing a translation

We base this recommendation on the APA recommendations article, Section Book, republished in translation. We recommend including the original title in English, following this APA suggestion. See the example of Piaget (1950).

The general format is:

[6] Yanina Bellini Saibene and Natalia Soledad Morandeira. Multilingual Data Science: Ten Tips to Translate Science and Tech Content. Chapter at Our Environment.A collection of work by data designers, artists, and scientists. ISBN:979-8-218-20191-3.

{list of authors of the English version}.{(year of publication)}. {Translated title}. {[Original Title in English]} (Translation to {language name}: {list of people that translate the material}). {DOI}. (Original work publish at {year of publication})

The text *Translation to* and *Original work publish at* should be written in the language of the translations.

Example using the developer guide and the Spanish translation:

> rOpenSci, Anderson, Brooke, Chamberlain, Scott, DeCicco, Laura, Gustavsen, Julia, Krystalli, Anna, Lepore, Mauro, Mullen, Lincoln, Ram, Karthik, Ross, Noam, Salmon, Maëlle, Vidoni, Melina, Riederer, Emily, Sparks, Adam, & Hollister, Jeff. (2021). Paquetes rOpenSci: Desarrollo, mantenimiento y revisión por pares [rOpenSci Packages: Development, Maintenance, and Peer Review] (Traducción al español: {name of the translators}) Zenodo. https://doi.org/10.5281/zenodo.6619350 (Trabajo original publicado en 2021)

2.4.2 Mention your contribution to a localization project in your CV

- <Start date ex.: May, 2022 – End date, ex: July, 2020>. Collaborative localization to Spanish of the book). General edition: Yanina Bellini Saibene. Contribution as <role, ex.: reviewer>. More information

2.5 References.

3 Translation and Review Guidelines

3.1 How to contribute

3.1.1 With an active language

You can check the list of languages for which there are active translation projects and their statuses here. There you will find the name of those who manage that language so you can get in touch and links to the active projects in that language. Each translation project also has a channel on the rOpenSci Slack where translation decisions are discussed and made, which are reflected in the language-specific guidelines (here) in this guide and a glossary.

Before you start, it is important that you familiarize yourself with the source material. In addition, we suggest that you read this guide (in particular the guidelines applicable to your language (here)) and contact the people who are already working on it. Do not hesitate to ask any questions you may have, for example, how much time do you have to spend on it? or technical questions like what if I run into problems on GitHub?

3.1.2 With a new language

If your language is not among the languages with active projects listed here, please contact the rOpenSci team to propose it. You can send an email to info@ropensci.org or open a *issue* in this repository.

Please note that starting a new project requires a commitment to manage the repository where all the review will take place. We ask from those who manage translation projects the same commitment we ask from those who manage packages of our suite: 2 years of maintenance or we can find someone else to take over.

Also, since each chapter or article must be reviewed by at least 2 people, we recommend you to invite your local community before starting the process.

3.2 Technical aspects of the review

In this section we will use the rOpenSci Development Guide as an example, but all the technical aspects of doing a review also apply to other rOpenSci materials.

The source code that generates the Development Guide lives at github.com/ropensci/dev_guide, and is organized in .Rmd (RMarkdown) format files that contain text and sometimes also code. In this case there are also some configuration files that need to be translated so that the guide in the new language is fully translated.

Each chapter has one .Rmd file per language, with a suffix that identifies the language using its two-letter code (according to ISO 639-1). For example, the chapter "Software Peer Review Policies" will have its original English version in `softwarereview_policies.Rmd` and its translated Spanish version in `softwarereview_policies.es.Rmd`.

If the translation project is running, you will find a series of *pull requests*, **one for each file to translate**, generated with the babeldown package containing the automatically translated text. **Each of these *pull requests* has an associated *branch*, this will be important later. In fact if there are many translation projects going on, you may encounter many *pull requests*. We recommend you to filter using the tag associated to your language, for example the tag for Spanish translation is "traducción ".

For security and organization, the reviewers will not have writing permissions in this main repository. So whoever manages the project will have to make a *fork* (which we will call **working repository**) and give write permission to all the people who review or contribute. This *fork* will have the same *branches* as the main repository; this is key since the **revision of each file will be done on the *branch* associated to its translation.

To start contributing the first thing you should do is to clone the working repository. To find out where it is located, talk to the person in charge of the project or consult the Slack channel. Once you have the work repository cloned, you can start working locally on your computer.

You're all set to start a review, but where do you start? As in any collaborative project, it is important to agree among all the people to know what each one is going to do. Again, we suggest you chat in the Slack channel to ask how the revisions will be divided.

3.2.1 Review 1

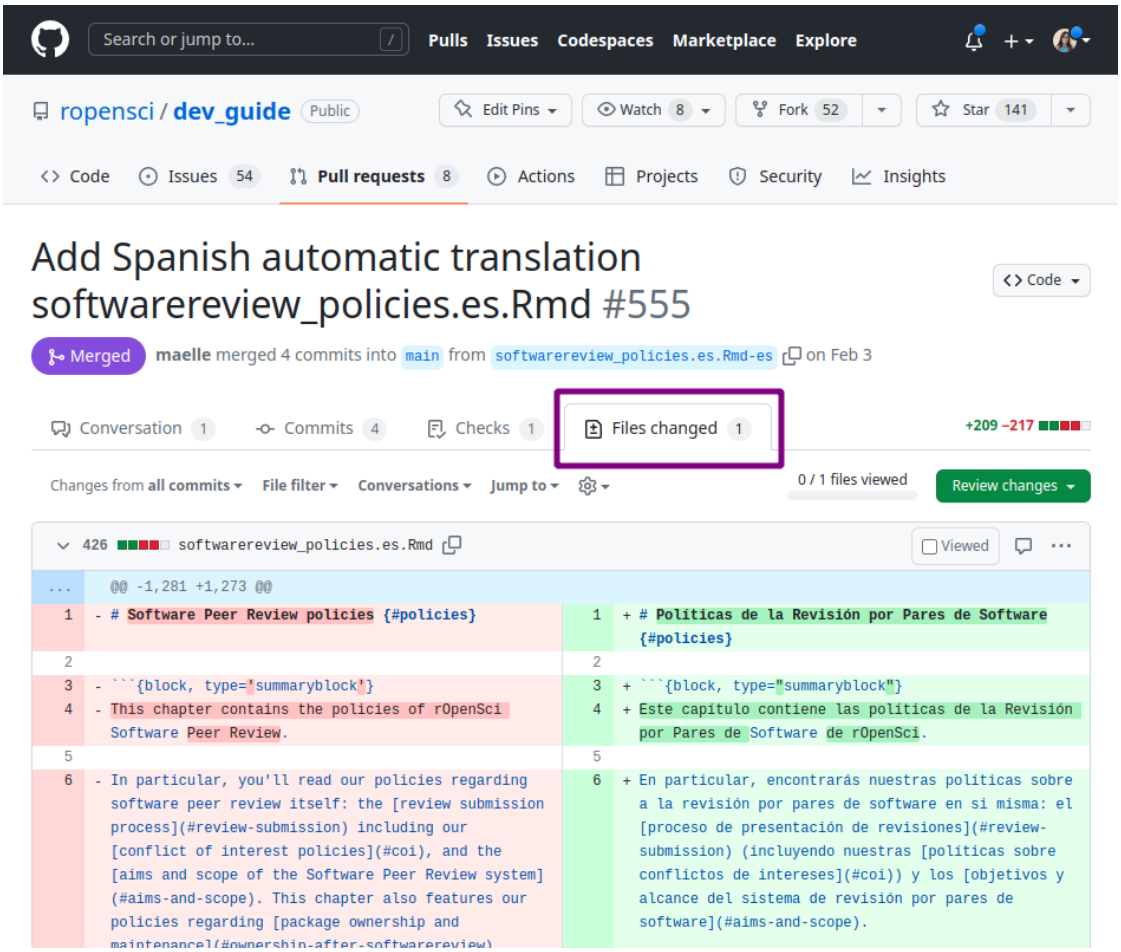
Let's say you had to do revision 1 of the chapter "Software Peer Review Policies" found in the `softwarereview_policies.en.Rmd` file. Here are the steps to follow.

In your local repository:

1. Activate the *branch* associated with the file you are going to review. In our example it would be `softwarereview_policies.es.Rmd-en`.

2. Open the `softwarereview_policies.es.Rmd` file. You may see things to improve immediately, as the automatic translation does a good job but it is not perfect.

3. Start reviewing the text. For this we suggest you have the English version next to you, you may need to go back to the original version to understand the context of some phrase or word that the translator has not been able to translate correctly. For this you can use the web version of the material you are translating or the *"Files changed"* tab of the PR in the main repository.



4. Periodically, or when you complete the revision, you will have to make a *commit* with the changes you generated. Many times the content of the files will be extensive and you will need a lot of time to complete the revision. In these cases we suggest you make *commits* at least after each work session indicating in the message up to which line of the file you have reached. This way, if you need to delegate the review to someone else, that person can review the file history and pick up where you left off.

5. So far the revision only lives in your local repository, the next step is to do a *push* to the

working repository. It is very important that the *push* is to the *branch* associated to the file you are working on, otherwise we will not be able to connect your revision with the original translation.

6. Once you have completed your revision, it is important that you notify the project coordinator or the person assigned to revision 2, if there is one.

3.2.1.1 Objectives of revision 1

The purpose of this first revision is to make a detailed reading of the machine translation in order to :

- Ensure that the meaning of the sentence or paragraph is maintained.
- Check that the glossary terms agreed upon by the community have been used according to the context.
- Apply stylistic decisions whenever possible. For example, in Spanish we decide to avoid gender marking of nouns and adjectives by paraphrasing sentences or using "las/los" if the former is not possible.
- Check that code blocks or Markdown markup have not been affected.
- Translate variable names and comments visible in the code blocks whenever appropriate.

Surely there are still some doubts and it is possible that some things may be overlooked. Style decisions are often the most difficult to implement and sometimes it is necessary to consult with the rest of the team or discuss it later with the reviewer 2.

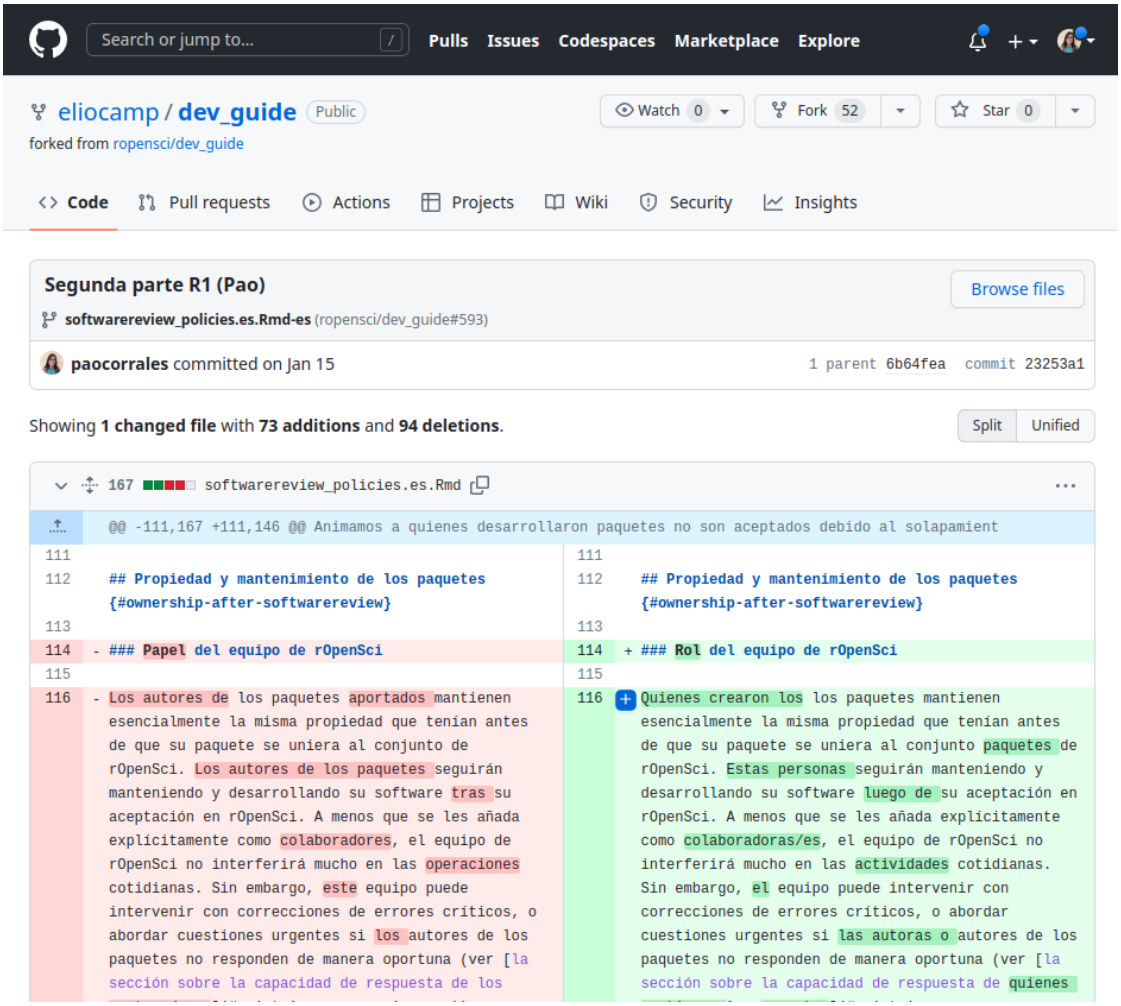
3.2.2 Review 2

You got revision 2, great. Again we will use as an example the chapter "Software Peer Review Policies" found in the `softwarereview_policies.en.Rmd` file. Whoever did review 1, whom we will call **R1** of this chapter most likely generated one or more *commits* with changes over the original translation. This means that your work does not start from scratch, here are the steps.

In your local repository:

1. Activate the *branch* associated to the file you are going to check. In our example it would be `softwarereview_policies.es.Rmd-en`.
2. Do a *pull* to download all the changes R1 made and have the file updated.

3. Start reviewing the text. To do this, we suggest you open the file history to view the changes R1 made. This will allow you to know if a phrase or word comes from the machine translation or from another human being.



4. Periodically, or when you complete the revision, you will have to make a *commit* with the changes you generated. Many times the content of the files will be extensive and you will need a lot of time to complete the revision. In these cases we suggest you make *commits* at least after each work session indicating in the message up to which line of the file you went. This way, if you need to delegate the review to someone else, they can review the file history and pick up where you left off.

5. So far the revision only lives in your local repository, the next step is to do a *push* to the working repository. It is very important that the *push* is to the *branch* associated to the file you are working on, otherwise we will not be able to connect your revision with the original translation.

6. Once you have completed your revision, the chapter is ready! However, this new version only lives in the working repository. You will have to open a *pull request* in the main repository to have it reviewed and approved by someone in an editor role. In the next section we will see how to open the *pull request*.

3.2.2.1 Objectives of Review 2

This second revision seeks to generate a text that maintains the original content but also sounds natural in the new language. In order to achieve this, the following must be taken into account:

- That the use of terms according to the glossary and the specific context is respected.
- Use the format defined by the community, for example, use *italicized* format for words that remain in English.
- Apply style decisions missed by R1 or missed by R1. For example, in Spanish try to replace any "las/los" with a phrase without gender marking.
- That, in general, sentences are understood and sound good while avoiding literal translations.

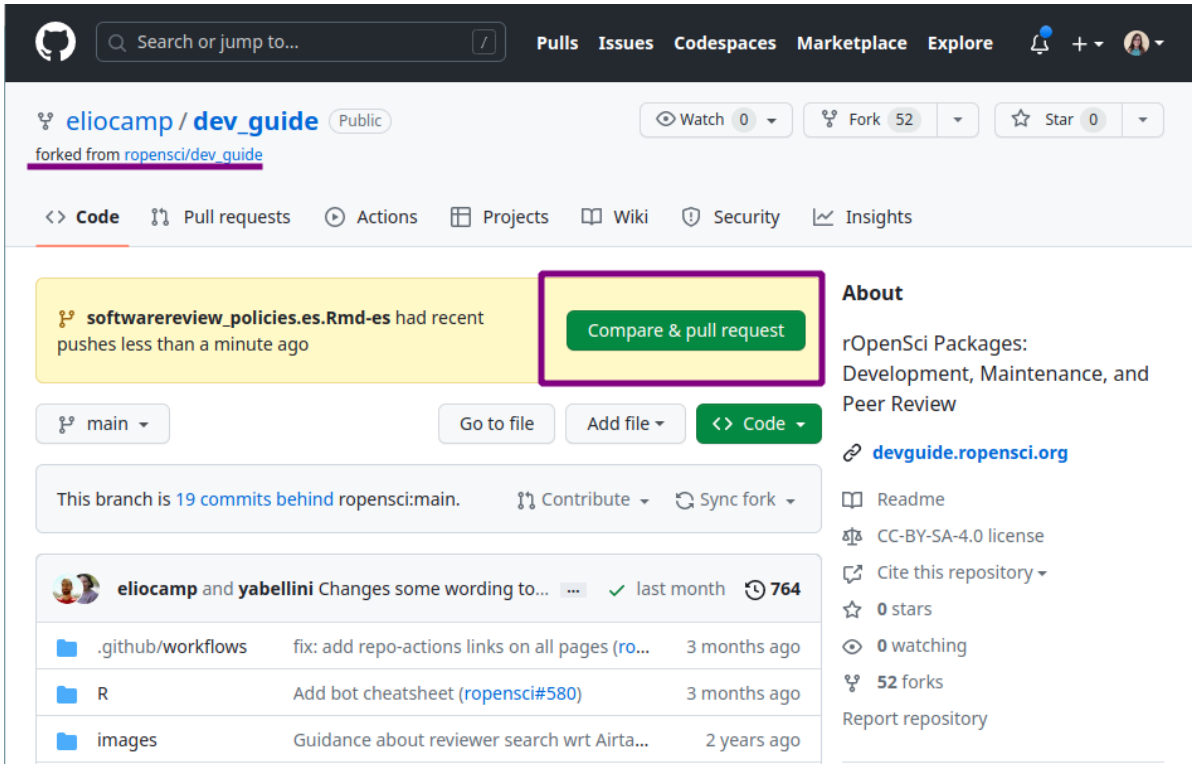
In the process of revision 2 you may need to consult with R1 about the decisions he/she made or with the rest of the community if there is a phrase or term that raises doubts. You may then suggest adding new terms to the glossary or updating this translation guide.

3.2.3 Pull Request and editing of a chapter

Once the process of revising a chapter is complete, it is time to transfer that new version from the working repository to the main rOpenSci repository. For this **R2** will have to open a *pull request* that compares the revised version with the machine translation only version.

1. Make sure that all changes are in the working repository and that you are working on the *branch* associated to the file under review, in our example it will be `softwarereview_policies.es.Rmd-en`.

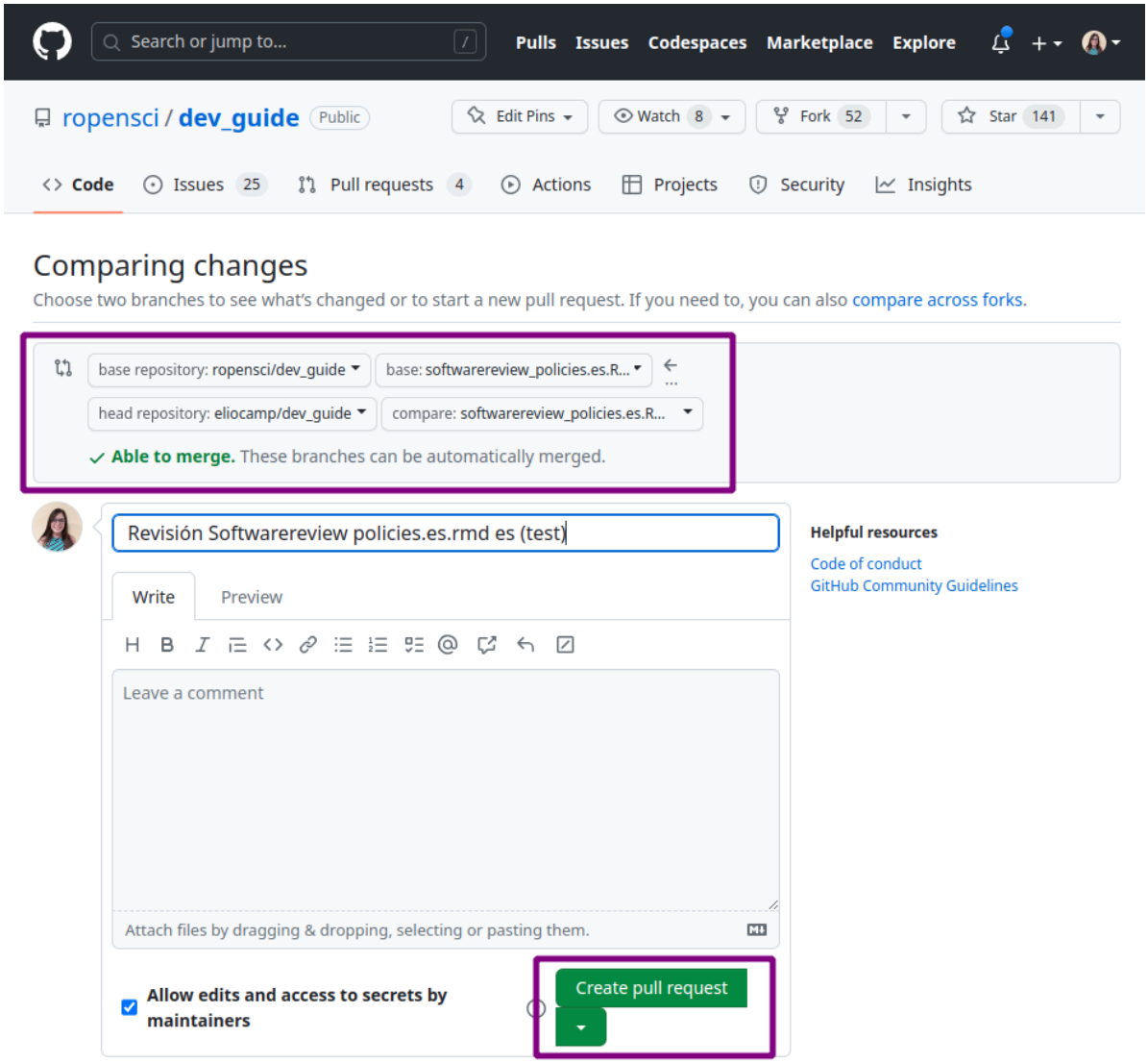
If you are on the correct *branch*, in the GitHub web interface you will see the following:



GitHub identified that there are new changes (*pushes*) in the *branch* and suggests comparing them and making a *pull request* to the main repository.

3. Click on the "*Compare & pull request" button.
4. You will then have the option to modify the title of the *pull request* and leave a message. We suggest 2 things:

    1. Change the title to identify that it is a "Review". It is important to separate it from the *pull request* of the automatic translation.
    2. In the body of the message, mention who is maintaining the translation project so that he/she is aware of the progress and can then continue with the process.



5. Before creating the *pull request* it is **very important** that you check that it is done from the correct *branch*. This is the only way to connect the review work with the automatic translation and to be able to unify the versions correctly.
6. And that's it! Click on "*Create pull request".

From this point on, the review is in the hands of the **maintainer** of the translation project. This person must review the *pull request* and make queries to **R1** and **R2** before *merging* the *pull requests.

3.3 General review

The revision process is a collaborative process in which, we hope, many people will collaborate. This has the great advantage of incorporating different views on the use of language and generating a translation that represents the greatest number of people. However, with all this diversity of views and despite the specific guidance of each language, it is possible that the revisions are heterogeneous, that sometimes different criteria are used or that there are simply typing errors.

For all these reasons, it is important that the translation project is completed with a global revision of all the material. We strongly suggest that this overall review be carried out by one person in order to maintain a single criterion throughout the text.

Now the translation is ready. Thank you and congratulations!

4 Specific Guidelines

> **Note**
>
> If you want to read the guidelines for a specific language, place change the version of the Translation Guide to that language using the language list on the left.

4.1 Technical terms and citations

1. Maintain a list of terms that are translated and another list of terms that are not. We propose a initial list and we consult our community (Slack first, social networks when there is not agreement).
2. For bibliographic references, the original title/name is left in English with the words in italics; a translation of the title in Spanish is added in parentheses. When a reference to a Wikipedia entry, a Carpentries lesson, or other online resource occurs, the Spanish version is added to the link if it exists. If one cannot be found, the reference is left in English.

4.2 Code

Translate:

1. variables names;
2. constant names;
3. comments;
4. user function names and parameters names: if a code snippet creates a function as example, we can name it and their parameters in Spanish.

   example:

```
Original code:

   inside <- function(point, lower, higher) {
        if (point <= lower) {
            return false
        } else if (point >= higher) {
            return false
        } else {
            return true
        }
   }

Translation to Spanish:

  adentro <- function(punto, mas_bajo, mas_alto) {
        if (punto <= mas_bajo) {
            return false
        } else if (punto >= mas_alto) {
            return false
        } else {
            return true
        }
   }
```

4.3 Diagrams

1. Figure and diagrams are translated.
2. Screenshots are translated only if the IDE or UI exist in the language of the translation.

5 NEWS